

Temporal Exchange — Engine Changelog for the CTO / Go backend

From your build `md5 80f050e2` (2026-06-14) → current HEAD `md5 bb2f8230` (2026-07-07). Feature-level, with a **Port** column = does the Go backend need a math/logic change, or is it display-only.

How to read this: everything under "Core math" and "Close & settlement" changes the numbers your backend must reproduce. "Display only" rows are UI — port only if you mirror the front-end. The gate suite `engine/verify/lens_selfcheck.js` (31 checks) doubles as your acceptance tests — every anchor number below is asserted there.

1. Core math (port these first)

#	Change	Plain English	Port
1	ITM option value rebuilt (linear smooth-paste, "PKG-ITM v2")	Your build's in-the-money values were wrong: the value curve dipped <i>below</i> the exercised payoff from $-S/K$ 0.80 down, and the exercise boundary sat at 0.444K instead of the correct 0.667K ($y=2$ put). Now the waiting-value power curve is welded <i>tangentially</i> (same value & slope) onto the LINEAR payoff line. Put boundary $S^*=K \cdot g / (g+1)$, call $K \cdot (g+1) / g$, $g=m \cdot \gamma$; boundary value $1 / (g+1)$; past it the value IS the payoff line. Guarantee (machine-checked + hard gate): quoted value never below the exercised payoff. Test anchors: $y=2, K=\$100, m=1 \rightarrow S^*=\66.67 , boundary $\frac{1}{3}$, ATM 0.148; $m=3 \rightarrow S^*=\$85.71$, boundary $1/7$, ATM 0.057.	YES
2	Trades conserve at the TRADE POINT	Your build applied conservation at the reserve point (a repeated spec regression). Now a trade executes where the strike ray meets the curve: $x_T = x \cdot p^{(w-1)}$, $y_T = y \cdot p^w$ ($p = tx\text{-ray}/\text{mode}$); the local pair $\alpha_T = w \cdot x_T$, $\beta_T = (1-w) \cdot y_T$ is conserved; $\Delta x = -\alpha_T \cdot \beta_T \cdot \Delta y / ((y_T - \beta_T)(y_T + \Delta y - \beta_T))$; new $w' = \alpha_T / (x_T + \Delta x)$. Anchor: $(x, y, w) = (10, 10, \frac{1}{2})$, ray $\theta=4$, $\Delta y=1 \rightarrow \Delta x = -5/22$, $x'=215/22$, $w'=\mathbf{11/21}$ exactly (not 22/43, not 6/11). Spot swaps ($p=1$) reduce exactly to your old formula — spot/arb/rebase code byte-identical. Consequence: α, β genuinely move on off-ATM trades by design.	YES
3	Vol knob: constant slope-multiplier m (if not already in your line)	Single scalar knob: lensed option-value steepness $g_{\text{loc}} = m \cdot \gamma$, constant at every strike ($m=1$ = plain curve; bigger m = steeper everywhere + trade lands further out via $\theta_{tx} = \text{mode} \cdot (\text{chosen}/\text{mode})^m$). Read and written through one shared helper; pool stays plain Balancer.	YES

2. Close & settlement — NEW in update-1 (2026-07-07)

#	Change	Plain English	Port
4	Close is ONE path — a sell-back trade for every leg	The old close had two branches: OTM legs reversed on the AMM, ITM legs were pulled off and cash-settled — a seam that made total payout JUMP ~45–87% at the OTM → ITM boundary. Now EVERY leg closes as a live reverse trade on today's curve (<code>tradeUpdateAt</code>), ITM included, no cash-settle branch. Payout is continuous across the strike (measured: crossing step $0.48 \times \text{median}$ = no jump). The trader is credited option/perp VALUE only ($L0 \cdot \text{raw_net} \cdot \text{carvedEquity}$, <code>raw_net</code> = option values locked <i>before</i> the swap) — the AMM swap is pool-internal bookkeeping, never credited to a wallet.	YES
5	Funding charged on the	Funding weight <code>mark</code> → <code>mark - intrinsic</code> (intrinsic = parity arm: put $\max(0, 1 - \text{mode}/\theta)$, call $\max(0, 1 - \theta/\text{mode})$). Funding becomes a single hump peaking at-the-money and	YES

option-part (extrinsic), not full value	exactly zero past the free boundary S^* (deep ITM). Reason: an ITM option is mostly the underlying, funded by the perp layer — don't double-charge. The pool-imbalance sign $\pm g \cdot (S-1)/S$ is unchanged. Note: on an equilibrium pool a pure spot move gives zero funding (oracle cancels); funding responds to pool disequilibrium.
---	---

- 🚫 **Residual x-drain — READ before multi-party launch.** The new close does NOT round-trip pool reserves exactly (the old close did). A round trip leaves a small underlying shortfall ($\Delta x < 0$, $\Delta y = 0$ exact). Verified characterization:
- **Non-extractable by construction** — the trader gets option value only; no code path credits the swap's dx/dy to any wallet (credit path byte-identical across the change).
 - **Impermanent-loss-like / recovering** — if price leaves and returns, the residual recovers to a fixed small value independent of the excursion.
 - **Bounded, $\propto$ notional²** (e.g. -\$29 on a test band). Harmless in a single-user sim; a small LP-vs-cycler asymmetry in a multi-party pool.

PARKED — UPDATE-2 (not implemented): a no-free-money floor + counterfactual charge-back neutralizes this and restores the exact no-free-money guarantee. **Implement it before shipping this close to a shared/multi-party pool.** Design: `specs/FIX_close_b_receipt_charge_PARKED_2026-07-03.md`.

3. Display-only (no Go port unless you mirror the UI)

#	Change	Plain English
6	Chart-2 true-value X wings + %/\$ toggle	Both wings drawn OTM → ITM crossing at ATM; pool-quoted continuation solid, escrow-parity tail dashed; fraction/dollar toggle. Old peak-at-1 tent retired.
7	Vol-direction caption fix	MORE volatile asset ⇒ LOWER m (fatter wings). Your build said the opposite.
8	m clamped to [1,6]	Out-of-range m silently drove the lens out of range; now clamps with write-back.
9	Funding P/L column per position line	Portfolio shows accrued funding as a SIGNED P/L number (negative = line paid). Line P/L = base + funding×oracle. Ledger stores "trader pays" as positive — display negates it (get this sign right). Disclosure shown: displayed P/L includes accrued funding; cash at close settles ex-funding until the funding transfer layer ships.
10	Caption/label truth-ups	Invariant-Watch states the trade-point law (α/β move off-ATM by design); "% of escrow unit" → "fraction of escrow unit"; band preview animates the actual per-leg path.

4. Verification you can lean on

- **Gates** engine/verify/lens_selfcheck.js — **31 hard checks** (+ `a16_atm_gate.js` 5). Every anchor number above is asserted; each check is negative-controlled (a sabotaged engine fails exactly the intended check). Use them as your Go acceptance tests.
- **CM6-v2 retired:** the old "exact round-trip / no-free-money" gate is gone (the new close is a pool-reprice close). No-free-money returns with the parked update-2 floor.
- **Formal (Lean, via Harmonic's Aristotle; artifacts on request):** value ≥ payoff, the seam weld + uniqueness, trade/rebase commutation.

5. Ruled but NOT in this handover (so nothing surprises you)

- Update-2: charge-back / no-free-money floor + LP & multi-wallet defenses (parked — needed before multi-party).

- Funding cash transfer between clubs (rate law exists; transfer layer is future).
- Read-layer smoothing (EMA-banded y) + skew rate-limit (calibration campaign, future).

Latest engine file: `engine/builds/HEAD_temporal_mvp_v28_lens.html` (md5 `bb2f8230`). Prior close retained as the revert twin `temporal_mvp_v28_lens_twocaseclose.html` (`51342574`). Generated 2026-07-07.